

Wczytywanie danych i zapis danych do plików i pliki nagłówkowe

1. Napisz funkcję `void printProdSimple(Produkt *p)` do uproszczonego drukowania elementów struktury `Produkt` z poprzednich zajęć w jednej linijce:

```
Produkt p1 = {"kreda", 0, 5.35}
printProd(&p1);
// wyświetli:
// kreda 0 5.35
```

2. **Odczyt plików tekstowych** Odczytaj z pliku `magazyn.txt` dane o wszystkich produktach. Wyświetl na ekranie każdą linijkę pliku. Policz przy okazji, ile jest linijek w `magazyn.txt`.
3. **Odczyt z pliku do bazy danych** Zadeklaruj tablicę na `Produkt`-y (ma pomieścić tyle, ile jest linijek w `magazyn.txt`). Wypełnij tablicę strukturami `Produkt`, które będą przechowywały dane z pliku `magazyn.txt` (odczytaj plik linijka-po-linijce i od razu zapisz dane z danej linijki do tablicy). Dobrze wykorzystać funkcję `przypiszDaneProduktu` z poprzednich zajęć.
4. **Modyfikacja rekordów w bazie danych** Po wczytaniu całej zawartości `magazyn` do tablicy, program ma zapytać użytkownika, jaki produkt chce kupić oraz ile sztuk (pomijamy już kwestię ceny). Program ma sprawdzić, czy mamy na magazynie dany produkt, o który pyta użytkownik w liczbie sztuk, o którą prosi.
 - jeśli tak - zmniejszyć zapisaną liczbę sztuk danego produktu w tablicy struktur o tyle, ile użytkownik chce kupić
 - jeśli nie - zakończyć pracę programu z kodem 1 (`return 1`)
5. **Zapis magazynu do pliku po modyfikacji** Zapisać w pliku `magazyn_update.txt` dane produktów na koniec - po możliwym pobraniu jakiejś liczby sztuk danego produktu przez użytkownika w poprzednim punkcie.
6. **Filtrowanie jedno- i wielokryteriowe:** chodzi o wyświetlenie na ekranie tylko tych produktów, które spełniają określone kryteria:
 1. wersja podstawowa: użytkownik podaje minimalną liczbę sztuk dostępną na magazynie. Pokazać wszystkie produkty według tej specyfikacji.
 2. rozszerzenie: użytkownik podaje minimalną liczbę ORAZ maksymalną cenę sztuki produktu. Pokazać wszystkie produkty, które spełniają OBA kryteria naraz.
 3. Finalna wersja: pytamy użytkownika:
 1. Chcesz filtrować po liczbie sztuk? (1/0)

2. Chcesz filtrować po cenie? (1/0)
3. W zależności od odpowiedzi użytkownika, pokazujemy listę produktów która spełnia jedno albo dwa kryteria (tak jak chciał użytkownik).
4. **TIP:** można to zrobić na wiele sposobów. Najprostsza logika na jaką pozwala nam `c` to prawdopodobnie coś takiego:
 1. Bierzemy po kolei każdy produkt.
 2. Domyślnie ustalamy flagę `pokaz_produk` na `1`.
 3. Jeśli użytkownik chciał filtrować po liczbie sztuk ORAZ liczba sztuk danego produktu jest mniejsza niż chciał, ustalamy flagę `pokaz_produk` na `0`.
 4. Jeśli użytkownik chciał filtrować po cenie i cena nie produktu nie przechodzi testu, też ustalamy flagę `pokaz_produk` na `0`.
 5. Wyświetlamy produkt tylko jeśli flaga `pokaz_produk==1`.

Aplikacja konsolowa z powyższego kodu (do domu, nie na kolokwium - ale może być na egzaminie)

Zrobić aplikację konsolową, która ma działać tak jak ten program:

1. Przekopiować cały kod do nowego pliku, np. `storage_app.c`.
2. Przenieść: deklarację `typedef`, deklaracje funkcji: `printProd` oraz `przypiszDaneProduktu` poza główny plik (`storage_app.c`) z kodem programu (zawierający funkcję `int main`) do pliku `prod.h` [przykładowa zawartość](#).
3. Do pliku `prod.c` wsadzić faktyczne definicje funkcji - zawrzeć `prod.h` za pomocą `include` (wszystkie pliki muszą być w tym samym folderze), [przykładowa zawartość](#).
4. Zawrzyj nagłówek `prod.h` także w pliku `main` (`storage_app.c`) za pomocą instrukcji `#include`. W `storage_app.c` mają być same dyrektywy `#include` i funkcja `main`. [szkielet zawartości](#).
5. Skompiluj kod za pomocą wiersza poleceń. Instrukcja dla Windowsa i CodeBlocks:
 1. Otwórz folder, w którym znajdują się pliki w wierszu poleceń (w eksploratorze plików, Prawy Przycisk Mysz > otwórz w wierszu poleceń)
 2. Znajdź ścieżkę do kompilatora, którego używa CodeBlocks:

```
Settings > Compiler ... > karta "Search directories"
```

Sprawdź, czy w karcie `Toolchain executables`, Kompilator C to `gcc.exe` - jeśli tak, powinien on być w folderze z punktu nr 2. Jeśli tam nie ma żadnej ścieżki, dodaj potem do ścieżki `Compiler's installation directory` w `Toolchain executables` człon `\bin\gcc.exe` (patrz punkt następny).

3. Aby skompilować program `storage_app.c` korzystający z pliku nagłówkowego `prod.h` i funkcji w `prod.c`, trzeba wpisać:

```
& 'C:\CodeBlocks\MinGW\bin\gcc.exe' storage_app.c prod.c -o storage_app
```

...zakładając, że faktycznie mamy kompilator `gcc.exe` i jest w folderze `'C:\CodeBlocks\MinGW\bin\'` (pkt 2)

Można też wybrać sobie inną nazwę dla pliku `.exe`:

```
& 'C:\CodeBlocks\MinGW\bin\gcc.exe' storage_app.c prod.c -o inna_nazwa
```

4. Jeśli nie mamy błędów w kodzie: skutkuje to powstaniem aplikacji - nowego pliku `storage_app.exe`. Możemy uruchomić ją w taki sam sposób, jak np. kalkulator na poprzednich zajęciach:

```
.\storage_app.exe
```

5. Dla systemów Linux-podobnych jest znacznie prościej. Najczęściej, kompilator `gcc` jest już zainstalowany i jest dodany do zmiennej środowiskowej `PATH`, co oznacza, że w wierszu poleceń można dokonać kompilacji pisząc po prostu:

```
gcc storage_app.c prod.c -o nazwa_mojej_aplikacji
```